

PRODUCT REQUIREMENTS DOCUMENT

Smart Asset Management & Resource Allocation Platform

Document Version	1.0
Status	Final
Domain	Full-Stack Asset Management
Platform Type	Web Application
Target Users	Administrators & Resource Consumers

Prepared for: IIT Roorkee Cultural Council Hackathon
Team Size: Up to 3 Participants

1. Product Overview

1.1 Purpose

This Product Requirements Document (PRD) defines the full scope, functional requirements, non-functional requirements, system design expectations, and acceptance criteria for the Smart Asset Management and Resource Allocation Platform. It serves as the authoritative reference for the development team throughout the hackathon build cycle.

1.2 Problem Statement

Organizations managing large pools of shared physical resources — such as the Cultural Council of IIT Roorkee — currently rely on fragmented spreadsheets, manual registers, and informal channels. This leads to:

- Scheduling conflicts from double-bookings
- Resource underutilization due to lack of visibility
- Delayed returns and lack of accountability
- Operational inefficiencies from manual coordination overhead

The platform will replace these processes with a centralized, web-based system providing real-time inventory visibility, automated booking workflows, and operational analytics.

1.3 Product Vision

Build a robust, intuitive, and scalable full-stack Asset Management Platform that enables any organization to manage shared resources efficiently — from booking requests through to issuance, return tracking, and analytics reporting.

1.4 Scope

Scope Area	Details
In Scope	User auth, inventory CRUD, asset booking, approval workflow, issue/return tracking, analytics dashboard, borrowing history
Optional Scope	Notifications, audit logs, QR codes, asset health tracking, Docker deployment
Out of Scope	Mobile native apps, payment integration, multi-organization tenancy (for MVP)

2. Stakeholders & User Roles

Role	Responsibilities
Administrator	Manages the full asset catalog, processes booking requests, approves/rejects, issues and receives returns, accesses system-wide analytics and audit data
Resource Consumer (User)	Browses the asset catalog, submits booking requests, tracks request status, views personal borrowing history and active bookings

3. Functional Requirements

FR-01: User Authentication & Session Management

Priority: CRITICAL | Role: All Users

ID	Feature	Requirement
FR-01.1	User Registration	New users can register with name, email, and password. Email uniqueness is enforced.
FR-01.2	User Login	Registered users can log in with email and password. Invalid credentials return an appropriate error.
FR-01.3	Role-Based Access Control	The system distinguishes between Admin and User roles. Role-specific routes and UI elements are protected.
FR-01.4	Secure Session Management	Sessions are maintained via JWT tokens or server-side sessions. Tokens must expire and support logout.
FR-01.5	OAuth (Optional)	Social login via Google or GitHub may be supported as an enhancement.

FR-02: Inventory Management

Priority: CRITICAL | Role: Administrator

ID	Feature	Requirement
FR-02.1	Add Asset	Admin can create a new asset with: name, category, description, total quantity, and initial status.
FR-02.2	Edit Asset	Admin can update any field of an existing asset. Changes are immediately reflected in the catalog.
FR-02.3	Delete Asset	Admin can delete an asset. Deletion must be blocked if active bookings or issued quantities exist.
FR-02.4	Categorize Assets	Assets are tagged with a category (e.g., Camera, Audio, Lighting). Category-based filtering is supported.
FR-02.5	Quantity Management	System tracks total quantity, allocated quantity, and available quantity. Available = Total minus Allocated.
FR-02.6	Asset Status	Each asset has a status: Available, Partially Available, Unavailable, Under Maintenance.

FR-03: Asset Discovery & Booking

Priority: CRITICAL | Role: Resource Consumer

ID	Feature	Requirement
FR-03.1	Browse Catalog	Users can browse all available assets in a paginated or scrollable catalog view.
FR-03.2	Search & Filter	Users can search by asset name and filter by category, status, and availability.
FR-03.3	View Asset Detail	Users can view full asset details including description, available quantity, and current bookings.
FR-03.4	Submit Booking Request	Users can request one or more units of an asset for a defined start and end date/duration.
FR-03.5	Quantity Validation	The system must reject any booking request where the requested quantity exceeds the currently available quantity.
FR-03.6	Duplicate Prevention	A user cannot submit duplicate overlapping requests for the same asset while a pending request exists.

FR-04: Approval Workflow

Priority: HIGH | Role: Administrator + Consumer

ID	Feature	Requirement
FR-04.1	View Pending Requests	Admin sees all pending booking requests with applicant details, asset, quantity, and duration.
FR-04.2	Approve Request	Admin can approve a booking. Available inventory is decremented immediately upon approval.
FR-04.3	Reject Request	Admin can reject a booking with an optional rejection reason. Inventory is unchanged.
FR-04.4	View Active Allocations	Admin can view all currently approved and issued bookings with their due dates.
FR-04.5	User Status Tracking	Users can see their request status: Pending, Approved, Rejected, Issued, Returned, Overdue.

FR-05: Asset Issue & Return Management

Priority: HIGH | Role: Administrator

ID	Feature	Requirement
FR-05.1	Mark as Issued	Admin can mark an approved booking as physically issued. Status transitions from Approved to Issued.
FR-05.2	Return Processing	Admin can mark an issued asset as returned. Status transitions to Returned. Inventory is replenished.

ID	Feature	Requirement
FR-05.3	Due Date Management	Every booking has a due return date. System flags bookings past their due date as Overdue.
FR-05.4	Inventory Accuracy	All inventory counts must remain accurate and consistent through the full lifecycle: Pending, Approved, Issued, Returned.

FR-06: Analytics Dashboard

Priority: HIGH | Role: Administrator

ID	Feature	Requirement
FR-06.1	Summary Cards	Display key metrics: total assets, total bookings, active allocations, available inventory, overdue returns.
FR-06.2	Most Utilized Assets	Bar chart ranking assets by number of bookings or total booked days in a selectable time range.
FR-06.3	Utilization Rate	Pie or donut chart showing what fraction of total inventory is currently allocated vs. available.
FR-06.4	Booking Trends	Line graph showing booking request volume over time (daily/weekly/monthly).
FR-06.5	Overdue Tracking	Dedicated view listing all overdue returns with borrower information and days overdue.

FR-07: Borrowing History

Priority: MEDIUM | Role: Administrator + Consumer

ID	Feature	Requirement
FR-07.1	User History	Each user can view their own complete borrowing history: past requests, current bookings, and statuses.
FR-07.2	Active Bookings View	Users can see a dedicated view of their currently approved or issued bookings with return due dates.
FR-07.3	Admin System History	Admin can view system-wide activity log of all bookings across all users, filterable by asset, user, date, or status.

4. Optional / Enhancement Features

ID	Feature	Priority	Description
OPT-01	Notification System	LOW	Email or in-app notifications for booking status changes (approved/rejected), upcoming due dates (24h before), and overdue alerts.
OPT-02	Audit Logs	LOW	Immutable log of system actions: asset creation/edits, booking approvals, returns. Viewable by admin with timestamp and actor.
OPT-03	QR Code Operations	LOW	Admin can generate a QR code per asset. Scanning the QR code during issue/return auto-fills the asset field. QR code links to asset detail page.
OPT-04	Asset Health Tracking	LOW	Track asset condition (Good / Damaged / Under Repair). Log maintenance events and damage reports per asset.
OPT-05	Dockerized Deployment	LOW	Provide Dockerfile and docker-compose.yml enabling single-command local environment setup.

5. Non-Functional Requirements

5.1 Performance

- All API responses must complete within 500ms under normal load conditions.
- The dashboard must render within 2 seconds including chart data.
- The system should support at least 50 concurrent users without degradation.

5.2 Security

- All passwords must be hashed using bcrypt or Argon2 before storage.
- JWT tokens must be signed and have a configurable expiry (default: 24 hours).
- All endpoints that modify state must require authentication.
- Role-based middleware must enforce Admin-only routes.
- Inputs must be validated and sanitized to prevent SQL injection and XSS.

5.3 Data Integrity

- All inventory quantity changes must occur within database transactions to prevent race conditions.
- Booking approval that would push available quantity below zero must be rejected at the data layer.
- Orphan bookings (for deleted assets) must be handled gracefully.

5.4 Usability

- The UI must be responsive and usable on modern desktop and tablet browsers.
- Error messages must be specific and actionable.
- Confirmation dialogs must be shown before destructive actions (delete asset, reject booking).

5.5 Maintainability & Scalability

- Codebase must follow a modular architecture (e.g., MVC or layered architecture).
- API must be RESTful with consistent naming conventions and HTTP status codes.
- Environment variables must be used for all secrets and configuration values.
- Database schema must support adding new asset categories and user roles without migrations breaking existing data.

6. System Architecture & Technology Stack

6.1 Recommended Architecture

The platform follows a standard three-tier architecture:

Layer	Technology Options	Responsibilities
Presentation Layer	React.js / Next.js or any modern frontend framework	Component-based UI, role-aware routing, chart rendering
Application Layer	Node.js + Express.js OR Python + Django/FastAPI	REST API, business logic, auth middleware, validation
Data Layer	PostgreSQL or MySQL (preferred) / MongoDB (acceptable)	Persistent storage of assets, users, bookings, history

6.2 Key System Components

- Auth Service: Registration, login, JWT issuance and validation, role management
- Asset Service: CRUD operations on assets, category and quantity management
- Booking Service: Request submission, validation against available stock, lifecycle state machine
- Analytics Service: Aggregation queries powering the dashboard
- Notification Service (Optional): Event-driven triggers for email/in-app alerts

6.3 Booking State Machine

State	Description
Pending	User submitted a request; awaiting admin review
Approved	Admin approved; inventory decremented; asset not yet physically handed over
Rejected	Admin rejected; inventory unchanged; user notified
Issued	Asset physically handed to user; due date active
Returned	Asset physically received back; inventory replenished
Overdue	Issued booking past due date and not yet returned

7. Database Schema Overview

Table	Key Columns
users	id, name, email, password_hash, role (admin/user), created_at, updated_at
assets	id, name, category_id, description, total_quantity, available_quantity, status, created_by, created_at, updated_at
categories	id, name, description, created_at
bookings	id, user_id (FK users), asset_id (FK assets), quantity_requested, start_date, end_date, due_date, status, rejection_reason, created_at, updated_at
asset_transactions	id, booking_id (FK bookings), action (issued/returned), performed_by (FK users), notes, timestamp
notifications (optional)	id, user_id (FK users), type, message, is_read, created_at
audit_logs (optional)	id, actor_id (FK users), action, entity_type, entity_id, metadata (JSON), timestamp

8. API Endpoint Overview

8.1 Authentication Endpoints

Method	Endpoint	Access	Description
POST	/api/auth/register	Public	Register a new user
POST	/api/auth/login	Public	Login and receive JWT
POST	/api/auth/logout	Auth	Invalidate session/token
GET	/api/auth/me	Auth	Get current user profile

8.2 Asset Management Endpoints

Method	Endpoint	Access	Description
GET	/api/assets	Auth	List all assets with search/filter query params
POST	/api/assets	Admin	Create a new asset
GET	/api/assets/:id	Auth	Get asset detail
PUT	/api/assets/:id	Admin	Update asset information
DELETE	/api/assets/:id	Admin	Delete asset (blocked if active bookings)
GET	/api/categories	Auth	List all asset categories

8.3 Booking Endpoints

Method	Endpoint	Access	Description
GET	/api/bookings	Admin	List all bookings (admin: all; user: own)
POST	/api/bookings	User	Submit new booking request
GET	/api/bookings/:id	Auth	Get booking detail
PATCH	/api/bookings/:id/approve	Admin	Approve a pending booking
PATCH	/api/bookings/:id/reject	Admin	Reject a pending booking
PATCH	/api/bookings/:id/issue	Admin	Mark booking as physically issued
PATCH	/api/bookings/:id/return	Admin	Mark booking as returned
GET	/api/bookings/history	Auth	Get borrowing history (user: own; admin: all)

8.4 Analytics Endpoints

Method	Endpoint	Access	Description
GET	/api/analytics/summary	Admin	Return summary card metrics (counts, utilization)
GET	/api/analytics/utilization	Admin	Asset utilization rates for charts
GET	/api/analytics/trends	Admin	Booking volume over time
GET	/api/analytics/overdue	Admin	List of all overdue bookings

9. UX & UI Requirements

9.1 Pages & Views

Page	Role	Key Elements
Login / Register	User	Auth forms with validation, role selection on registration
Asset Catalog	User + Admin	Card or table layout, search bar, category filter, availability filter
Asset Detail	User + Admin	Full asset info, availability calendar or count, booking form
My Bookings	User	List view: pending, active, history; status badges; due dates
Admin — Requests Queue	Admin	Pending requests table: approve/reject actions, bulk actions
Admin — Active Allocations	Admin	Approved and issued bookings; mark issued / mark returned actions
Admin — Inventory	Admin	Full asset list with CRUD; quantity indicators
Analytics Dashboard	Admin	Summary cards row + 3 charts: utilization pie, top assets bar, trends line
Admin — Overdue	Admin	Table of overdue bookings with borrower contacts and days overdue

9.2 UI/UX Principles

- Role-aware navigation: Admin and User see distinct nav menus with only their relevant items.
- Status badges with consistent color coding: Pending (yellow), Approved (blue), Issued (green), Rejected (red), Overdue (red/orange), Returned (gray).
- Empty states: All list views must show a helpful empty state message when no records exist.
- Loading indicators: Async operations must show a loading spinner or skeleton state.
- Form validation: All forms must show inline field-level error messages before submission.
- Confirmation dialogs: Required for delete asset, reject booking, and return asset actions.

10. Deliverables & Acceptance Criteria

#	Deliverable	Acceptance Criteria
D1	Working Application	All mandatory FRs (FR-01 to FR-07) are implemented and functional in the demo. No critical bugs blocking core workflows.
D2	Design Document (PDF)	8-page max PDF covering: problem understanding, system architecture, DB schema, ERD, API overview, design decisions.
D3	Public GitHub Repository	Source code, config files, DB scripts, and documentation committed. README present. Project is reproducible.
D4	README.md	Includes: project overview, tech stack, prerequisites, setup steps, running instructions, and feature list.
D5	Demo Video (3 min max)	Covers: auth, asset booking flow, approval process, dashboard, and key technical features. Clear audio/screen recording.

11. Engineering Guidelines & Best Practices

11.1 Code Quality

- All source files must follow consistent formatting (Prettier, ESLint, or equivalent).
- Functions and modules must have single, clear responsibilities.
- No hardcoded secrets — use .env files and provide a .env.example.
- Avoid deeply nested callback structures; prefer async/await.

11.2 Data Integrity Rules

- All inventory mutations (approve, issue, return) must use database transactions.
- Optimistic locking or row-level locking should be used when decrementing available_quantity to prevent race conditions.
- Asset deletion must CASCADE or SET NULL on dependent booking records according to the chosen strategy, documented in the design doc.

11.3 Testing Expectations

- Critical API endpoints (auth, booking submission, approval) should have at least basic unit or integration tests.
- The system should handle invalid inputs gracefully and return appropriate HTTP status codes (400, 401, 403, 404, 409, 500).

11.4 Documentation

- All API endpoints documented in README or a separate API.md.
- DB schema documented via ERD diagram in the design document.
- Any non-obvious design decisions must be justified in the design document.

Appendix: Feature Priority Summary

ID	Feature	Priority	Classification
FR-01	User Authentication	CRITICAL	Must Have
FR-02	Inventory Management (Admin)	CRITICAL	Must Have
FR-03	Asset Discovery & Booking (User)	CRITICAL	Must Have
FR-04	Approval Workflow	HIGH	Must Have
FR-05	Issue & Return Management	HIGH	Must Have
FR-06	Analytics Dashboard	HIGH	Must Have
FR-07	Borrowing History	MEDIUM	Must Have
OPT-01	Notification System	LOW	Nice to Have
OPT-02	Audit Logs	LOW	Nice to Have
OPT-03	QR Code Operations	LOW	Nice to Have
OPT-04	Asset Health Tracking	LOW	Nice to Have
OPT-05	Dockerized Deployment	LOW	Nice to Have

End of Document